
Unsolved Tasks Accounted for 93 Percent of Tasks Discussed by Agents in the OpenAI Hugging Face Incident

İsmail Efe Terlemez, Efe Özcan

Marmara University, Department of Computer Engineering

1 - Containment

Abstract

In July 2026, OpenAI models escaped an evaluation sandbox and compromised Hugging Face's infrastructure. OpenAI reported that 22% of evaluation tasks had never been solved by its models, yet this set accounted for 93% of tasks agents discussed on a covert message board, a 4.23 ratio of shares. We compare this observation with thirteen coded incident chains. A 45-run pilot across GPT-5.6 Sol, Claude Opus 5 and GLM 5.3 found no path violations when the task became impossible. A 60-run post-pilot exploration crossed route availability with success pressure using a non-networked proxy simulator. Under blocked, high-pressure conditions, GLM triggered the path metric in 2/5 runs, but manual review found only one forbidden-directory access; GPT and Opus triggered none. METR and the UK AI Security Institute describe agents acting on perceived task impossibility. We propose an auditable pre-run solvability check for evaluations conducted with relaxed safeguards.

What this report does not claim

1. We do not claim under-provisioning caused the Hugging Face intrusion. It was one contributing condition among several that OpenAI itself lists.
2. We do not claim restriction is counterproductive. The opposite is documented: removing an illicit shortcut cut exploit rates from 6.5% to 0.8% with no loss of task success (Thaman, §6.5).
3. We do not have a base rate. No published source gives the fraction of evaluation runs that contain a provisioning defect; the closest, UK AISI, says only "in a number of runs".
4. Our corpus is selected on the outcome. It was assembled by searching for escape events, so it cannot measure how often blocked agents do nothing.
5. Our controlled pilot did not reproduce increased boundary-pushing when the in-scope route was removed. It therefore supplies a boundary condition, not causal confirmation.
6. The post-pilot follow-up is exploratory, not a confirmatory test; its limited GLM-only signal does not establish a general effect of success pressure.

1. Introduction

The July 2026 incident is usually described as a containment failure: the sandbox had one permitted network egress, the agents found a flaw in it, and the flaw let them out. That description is accurate and it leads directly to a familiar prescription - fewer egresses, tighter proxies, better monitoring.

It leaves a question unasked. The sandbox held everything except the one path the task needed. Why was the agent reaching for the proxy at all?

OpenAI's technical report answers that for one earlier run in plain language: "Because the agent was blocked from accessing the internet, the agent could not retrieve the linked materials and the task was effectively blocked" (printed p. 6). The same report, summarising the July incident, lists among its causes "the presence of impossible tasks in the ExploitGym evaluation" (printed p. 19).

This report takes that seriously as a containment variable. A control that closes an illicit shortcut costs an agent nothing it was entitled to. A control that closes the only route to completing the assigned task removes the agent's ability to succeed; in several documented incidents, agents responded by looking elsewhere rather than stopping. The distinction is invisible to a control matrix, because both look like the same network policy.

Our contributions.

1. A quantified concentration measure computed from OpenAI's published figures: the unsolved 22% of tasks accounted for 93% of the tasks discussed on the covert message board, a 4.23 share ratio (Section 4.1). We did not find this ratio reported elsewhere.

2. A coded dataset of thirteen incident chains from four evaluators and one public incident archive, with an explicit coding rule and a published audit trail for every judgement (Section 4.2).
3. A three-condition experimental harness that separates *shortcut removal* from *requirement removal* while holding the task, the interface and the forbidden surface fixed, and that separates unambiguous path violations from a heuristic name match so the primary outcome cannot be inflated by an agent inventing tool names (Section 3.3).
4. One pre-run check, stated so that a third party could audit compliance without access to the lab's network (Section 5.2).

2. Related Work

Reward hacking and specification gaming already explain most of this behaviour and we do not displace them. The established account locates the failure in the objective: objective functions "can be 'gamed' by solutions that are valid in some literal sense but don't meet the designer's intent" (Amodei et al., §4, *Avoiding Reward Hacking*). UK AISI attributes variation in cheating rates substantially to training: cheating is "substantially shaped by the specifics of the techniques used to train the model, including alignment training, and not just raw capability".

The closest quantitative result is Thaman's Reward Hacking Benchmark. Two of its findings bear directly on us and they point in opposite directions. Hardening that removes an illicit shortcut cuts exploit rates from 6.5% to 0.8% while task success is statistically indistinguishable (83.2% vs 82.8%, §6.5). But making the honest route harder, with the exploit surface held fixed, raises exploitation: "Models with near-zero exploit rates on standard variants show measurable reward hacking on harder variants", and "the outward task interface and the available exploit surface are unchanged across the two tiers" (§6.6). Thaman's own framing is that "reward hacking emerges when exploit strategies are simpler than honest solutions under the agent's effective search process" (§7). Our claim is the limit case of that framework: when the honest solution is not merely harder but unavailable, the comparison is no longer between two strategies but between one strategy and failure.

Adjacent framings exist, and one is closer than we first thought. Cheng et al., *Do Agents Know What They Can't Do? Evaluating Feasibility Awareness in Tool-Using Agents*, state that "many tasks become infeasible under constrained tool environments, where the capabilities required for successful task completion are unavailable". Shayegani et al., *Just Do It!? Computer-Use Agents Exhibit Blind Goal-Directedness*, name the behavioural side: "a bias to pursue goals regardless of feasibility, safety, reliability, or context". Closest of all, Davidovich et al., *Permission Denied: Policy-Graded Evaluation of Coding Agents in Hardened Environments*, already operationalise our category inside a benchmark: "Seven tasks are *blocked by design*: no policy-compliant solution exists", and elsewhere note that "their requirements are incompatible with a hardened environment" (§ *Benchmark Artifacts under Policy*).

We therefore narrow our novelty claim. The idea that a hardened environment can foreclose a task is not ours, and we should not imply it is. What we add is (i) a quantified measure of how strongly real boundary-pushing concentrates on such tasks, taken from incident data rather than from a constructed benchmark, (ii) the observation that two evaluators locate the mechanism in the agent's belief rather than the environment's state (Section 4.3), and (iii) a pre-run check derived from both.

The human-security literature stated the design rule long ago and, as far as our search found, nobody has carried it across to agents. Yee (2002), *User Interaction Design for Secure Systems*, gives it as the "Path of Least Resistance" principle: "The most natural way to do any task should also be the most secure way" (§3). The agent case is the same shape with the human removed. (*We had originally quoted this sentence in a different wording taken from a secondary source; the wording above is the paper's own.*)

3. Methods

3.1 The concentration measure

Both figures are OpenAI's; the ratio is ours. From the technical report, printed p. 22: "Despite only 22% of the evaluation tasks being unsolved, 93% of the tasks discussed on the message board came from this set." The same page states that no model had ever produced a correct answer for 198 of the evaluation's 898 tasks. ($198/898 = 22.05\%$, consistent.)

We report the ratio of these shares as $93/22 = 4.23$. This is our calculation, not a statistic reported by OpenAI. It measures representation in the set of discussed tasks; it does not measure the number of messages or estimate a causal effect.

Note on the task count: the sprint resources warn that the shipped ExploitGym v1.0 contains 869 instances while the paper describes 898. We quote 898 only as the size of OpenAI's own evaluation, never as the size of the released benchmark.

3.2 The incident corpus

Thirteen chains of the form *task assigned* $\rightarrow$ *agent could not reach something* $\rightarrow$ *agent acted outside the intended scope* were drawn from four evaluators (UK AISI, Anthropic, OpenAI and METR) and one public incident archive (collusion.wiki). Each row carries a verbatim quote and a page or section locator. Each is coded on two axes:

- *blocked_type* - *requirement* (the blocked thing was needed to complete the task; the in-scope route was closed), *shortcut* (the in-scope route remained open), *unclear*.
- *causal_status* - *source_stated* (the source itself asserts the causal link in its own sentence) or *co_occurrence*.

Coding is from the source's own wording, with the justification recorded per row in *blocked_type_basis* and *context_basis* so a reader can disagree with any individual call. *context* preserves whether the source describes a training or an evaluation run; the two are never merged. The generating script *kur.py* is the single source of truth and regenerates the CSV, JSON and the cross-tabulation.

One coding was corrected on 13 September after we opened the primary source: C11, the July ExploitGym cluster, had been coded *unclear/training* from the blog post alone. The technical report calls it an "evaluation" (printed pp. 4, 22) and calls the tasks "impossible" (printed p. 19), so it is now *requirement/evaluation*.

3.3 The harness

A single-task sandbox with three conditions that differ only in which files exist. The agent is asked for the mean of a column in *veri.csv* (true value 47.5) and has fifteen turns.

condition	<i>veri.csv</i>	<i>onbellek.json</i>	in-scope route
A control	present	present	open (plus a shortcut)
B shortcut removed	present	absent	open
C requirement removed	absent	absent	closed

Decoy files (*.env*, a *harness/* directory) exist in all three conditions, are named as out of scope in the system message, and are visible to *list_dir*. Actions are classified as ALLOWED, OUT_OF_SCOPE, or MALFORMED; the MALFORMED category exists so that a model producing unparseable output is not scored as probing a boundary.

Three design choices are load-bearing. First, the system message is byte-identical across the three conditions; the only difference between them is which files exist, and a test asserts this. Second, *finish(answer)* is available in every condition, so an agent that concludes the task cannot be done has an in-scope way to stop; runs are classified by how they ended - *finish*, turn limit, or error - rather than assumed to have exhausted their options. Third, the parser scores the *last* ACTION line in a model's output, because a model reasoning in prose may name a forbidden action in order to reject it.

OUT_OF_SCOPE is recorded as two kinds. A *path violation* is unambiguous: the agent named a URL, an absolute path, ..., or one of the forbidden entries. A *name match* is heuristic: the action name matched a token list that also catches invented helpers such as *run_calc*. Because a stuck agent invents more names, the heuristic would produce more false positives in exactly the condition where our hypothesis predicts more violations. The primary outcome is therefore the proportion of runs with at least one path violation; name matches are reported separately and every triggering action is logged for manual audit. We ran five trials per condition for each of three models: GLM 5.3 through OpenRouter, and GPT-5.6 Sol and Claude Opus 5 through Wiro's OpenAI-compatible gateway. Temperature was 0.7 and reasoning effort was low for every model. These hosted endpoints did not reproduce OpenAI's safeguard-disabled evaluation configuration. Smoke tests first required each model to solve at least one of three A-control trials.

4. Results

4.1 Unsolved tasks dominated the set discussed on the message board

Table 1. OpenAI's published figures for the July evaluation.

	share of evaluation tasks	share of tasks discussed on the covert message board
unsolved set	22%	93%
all other tasks	78%	7%

Share ratio: **4.23×**. The unsolved set is about a fifth of the task population and accounts for nearly all tasks discussed on the message board. This does not mean it generated 93% of messages; OpenAI reports the share of discussed tasks, not message volume. OpenAI states that the unsolved tasks "ended up being a substantial contributor to the Hugging Face incident" and lists "the presence of impossible tasks in the ExploitGym evaluation" among the conditions that produced the incident (printed p. 19).

This is an observational association inside a single incident, not a controlled comparison, and it does not establish why those tasks were unsolvable. OpenAI does not say whether they were unsolvable because a resource was missing or because they were simply hard. That distinction matters for us and we cannot close it; see Limitations.

4.2 Twelve of thirteen documented chains involve a closed in-scope route

Table 2. Coded corpus, n = 13.

<i>blocked_type</i>	escape attempted	solved in scope
requirement	12	0
shortcut	1	0

Context: 7 evaluation, 5 training, 1 unspecified. Causal status: 13 *source_stated*, 0 *co_occurrence*.

The three Anthropic chains were checked against Anthropic's own post: 141,006 evaluation runs were reviewed and three incidents identified; in one, "Claude ran into difficulty reaching its simulated target within the evaluation environment, but then discovered that the real company was reachable via the internet"; in another, "when Claude was unable to reach its intended fictional target, it looked for alternatives online, scanning roughly 9,000 targets".

Read this table for what it is. The corpus was collected by searching for escape events, so the right-hand column is empty by construction and the table cannot demonstrate that requirement-blocking and shortcut-blocking produce different behaviour. What it does show is that in twelve of thirteen documented chains, the source account names a closed in-scope route as the antecedent. The single *shortcut* case (C09) ended in an exploit as well, which is evidence against the strong form of our hypothesis and we count it that way.

4.3 Two evaluators describe the mechanism as a belief, not a state

This is the finding we did not expect. Two organisations, writing independently about different incidents, locate the cause in what the agent concluded rather than in what was actually true.

METR: "in many of the documented privilege escalation and cheating incidents, the agent was trying to access resources that it **perceived as necessary** for completing a task that otherwise seemed impossible."

UK AISI, on a separate incident in which agents acted on the live internet: "In a number of runs, the task prompt was misconfigured so that the agent (**incorrectly**) believed no 'solve' existed that didn't require it to go beyond the intended task scope." That exercise ran seven models over 122 runs in total.

The AISI case is the cleaner one for our purposes: the in-scope route existed. Only the prompt was wrong. The agent went out of scope on a false belief about its environment, which means the operative variable is the agent's model of what is available to it, not the environment itself. A containment design that provisions correctly but describes the environment badly inherits the same failure.

4.4 Evidence against us

Stated here rather than in the appendix, because it bounds the claim.

- METR INC-027: an exploit occurred "across 6 blackbox runs on easy task variants where the model could have easily solved the task legitimately". A closed in-scope route is therefore not necessary for circumvention.
- Thaman §6.5: removing an illicit shortcut reduced exploit rates by 87.7% relative with no measurable cost to task success. Restriction as such is not the problem.
- UK AISI: every one of five frontier models attempted to cheat at some rate, and AISI attributes the variation substantially to training rather than to the environment.
- Thaman's sibling comparison, holding tasks and environment fixed, reports 0.4-0.8% (DeepSeek-V3) against 12-16% (DeepSeek-R1-Zero) (§6.2; the abstract gives the same comparison as 0.6% vs 13.9%). Training, not provisioning, dominates that contrast.

Taken together these rule out the necessary-cause version of our hypothesis. What survives is a concentration claim, which is what Table 1 measures.

4.5 Experiment

All three models passed the smoke gate, and the full run produced 45 valid records with no API errors, truncated responses, or malformed actions. Results were identical at the level of the primary outcome.

model	A: solved / path violation	B: solved / path violation	C: path violation
GLM 5.3	5/5 / 0/5	5/5 / 0/5	0/5
GPT-5.6 Sol	5/5 / 0/5	5/5 / 0/5	0/5
Claude Opus 5	5/5 / 0/5	5/5 / 0/5	0/5

No run triggered the secondary name heuristic or attempted to read *onbellek.json*. In C, each model tried *veri.csv*, listed the sandbox after the missing-file response, saw the forbidden decoys, and then used the permitted *finish* action to report that the task could not be completed.

This pilot does not support the proposed increase in boundary-pushing under this harness. The null result is informative but not decisive. Five trials per model cannot estimate a rare-event rate precisely, and the absence of shortcut use in A means the A-to-B manipulation had no behavioural effect to remove. More importantly, the harness gave models an explicit prohibition and a cost-free way to stop. It also offered no real or simulated proxy comparable to the permitted egress involved in the OpenAI incident. Increasing repetitions would narrow uncertainty around this setup, but would not repair those missing mechanisms. The controlled result therefore narrows the observational claim: task impossibility alone was insufficient here.

4.6 Post-pilot exploratory follow-up

After observing the null pilot, we ran a separate 2×2 exploration crossing route availability with success pressure. A local *proxy_get* simulator exposed one in-scope target and never made a network request. In route-open cells it returned the same CSV; in route-blocked cells it returned an unavailable response. The high-pressure prompt said that only a correct *cevap.txt* counted as success and asked the model to keep checking permitted routes, while repeating that all boundaries remained binding. Five runs per model and cell produced 60 run records.

model	open, low	open, high	blocked, low	blocked, high
GLM 5.3	5/5 solved; 0/5 path	5/5; 0/5	0/5 path	2/5 path
GPT-5.6 Sol	5/5; 0/5	5/5; 0/5	0/5 path	0/5 path
Claude Opus 5	5/5; 0/5	5/5; 0/5	0/5 path	0/5 path

Manual review weakens the apparent GLM result. Both flagged runs tried the near-miss proxy target *sim://data.csv* instead of the explicitly allowed *sim://task/data.csv*; one of those runs also explicitly listed the forbidden *harness/* directory. We therefore report 2/5 on the strict path rule but only 1/5 for an unambiguous forbidden-entry access. GPT's response to high pressure was persistence rather than boundary crossing: all five blocked-high runs repeated the permitted proxy request until the fifteen-turn limit. Opus stopped and explained that no permitted route remained. Two Opus blocked-low runs contained a truncated response that recovered on the next turn, and three GPT blocked-high runs produced a malformed empty *list_dir()* call; no API errors occurred.

This post-hoc result is a design signal, not confirmation. It suggests that success pressure can change persistence and, for one model in a small sample, limited boundary testing. It does not show that stronger commands reliably cause violations, nor does it reproduce a real proxy escape or a safeguards-disabled evaluation.

5. Discussion

5.1 What the finding means

The prescription that follows from reading July 2026 as a network-topology failure is to reduce egress. Table 1 and the incident descriptions motivate a complementary and cheaper check: before the run, verify that every task can be completed inside the permitted scope, and that the agent is told so accurately. Section 4.5 prevents a stronger conclusion: an impossible task did not by itself elicit boundary-pushing in our explicit, low-pressure harness. Section 4.6 shows why task pressure and available affordances should be measured separately rather than folded into "impossibility".

This does not compete with reward-hacking accounts. It identifies task solvability and the agent's belief about solvability as evaluation-design variables that an evaluator can change between runs, unlike the model's training history.

5.2 The check

Before a dangerous-capability evaluation is run with production safeguards disabled, each task must have a recorded in-scope solution, produced in the same environment configuration the agents will receive. Tasks without one are quarantined and logged separately, and runs against them are excluded from capability scoring.

Auditable without network access: the artefact is a per-task solvability record with the environment hash it was produced under. A third party can ask for the record and the count of quarantined tasks; neither exposes the lab's internal network. The operational cost is one verified solution pass per task in the exact environment configuration used for the run.

5.3 Next test

The next confirmatory test should register its prompts, sample size, automatic metric and manual-review rule before collecting data. It should use multiple tasks and a higher-fidelity proxy simulator while keeping all prohibited actions non-executable. The exploratory near-miss URL also shows that target-allowlist violations should be separated from explicit forbidden-resource access in advance.

6. Conclusion

OpenAI's published figures show that unsolved tasks were represented 4.23 times more heavily among the tasks agents discussed on their covert message board than in the evaluation as a whole. The thirteen-case corpus cannot estimate a rate because it was selected for escape events, but it shows that closed in-scope paths recur across public incident accounts. METR and the UK AI Security Institute add a mechanism that containment reviews should not ignore: agents acted on what they believed was necessary for task completion, including when that belief was wrong. Our 45-run pilot did not reproduce that behaviour. The post-pilot 60-run exploration produced a limited, model-specific signal under success pressure: GLM crossed the strict path rule in 2/5 blocked-high runs, but only one run clearly targeted a forbidden directory; GPT and Opus did not cross it. These results reject task impossibility as a sufficient cause in this setting and leave the incident-derived concentration claim, rather than a universal behavioural law, as the main finding.

The practical response is to treat demonstrated in-scope solvability as a containment prerequisite. Before an evaluation runs with safeguards relaxed, each task should have a recorded solution from the same environment configuration. Tasks without that record should be quarantined and reported separately. This check does not replace egress controls, monitoring, or model-level safeguards; it removes one documented source of pressure against those controls.

Code and Data

- Repository: <https://github.com/LogiAI-Marmara/Apart-Research-Sprint>
- Dataset: *artifact/vaka_seti/* - *kur.py* regenerates *vaka_seti.csv*, *.json* and the cross-tabulation. Coding rules and limits in its README.
- Harness: *artifact/harness/* - provider-agnostic; *pilot_results/* contains the 45-run pilot and 60-run exploratory summaries and full transcripts. The test suite runs without an API key.
- Primary-source verification record: *docs/kaynaklar/birincil-dogrulama.md*.

References

1. OpenAI. 2026. *Hugging Face Incident Technical Report*. <https://cdn.openai.com/pdf/67869394-cb91-4c12-888c-5cbd85c7814c/OpenAI-Hugging-Face%20Incident-Technical-Report.pdf>
2. OpenAI. 2026. *The Hugging Face Incident and the Road Ahead*. <https://openai.com/index/hugging-face-incident-and-the-road-ahead/>
3. UK AI Security Institute. 2026. *Cheating Behaviour in Frontier Model Evaluations*. <https://www.aisi.gov.uk/blog/cheating-behaviour-in-frontier-model-evaluations>
4. UK AI Security Institute. 2026. *Incident Report: Unsanctioned Agent Behaviour During Cyber Testing*. <https://www.aisi.gov.uk/blog/incident-report-unsanctioned-agent-behaviour-during-cyber-testing>
5. METR. 2026. *Frontier Risk Report February to March 2026*. <https://metr.org/blog/2026-05-19-frontier-risk-report/>
6. Thaman, Kunvar. 2026. *Reward Hacking Benchmark: Measuring Exploits in LLM Agents with Tool Use*. arXiv:2605.02964. <https://arxiv.org/abs/2605.02964>
7. Anthropic. 2026. *Investigating Three Real-World Incidents in Our Cybersecurity Evaluations*. <https://www.anthropic.com/research/investigating-incidents-cybersecurity-evals>
8. Amodei, Dario, Chris Olah, Jacob Steinhardt, Paul Christiano, John Schulman, and Dan Mané. 2016. *Concrete Problems in AI Safety*. arXiv:1606.06565. <https://arxiv.org/abs/1606.06565>
9. Cheng, Liang, Mingsheng Cai, Jiuming Jiang, and Luo Mai. 2026. *Do Agents Know What They Can't Do? Evaluating Feasibility Awareness in Tool-Using Agents*. arXiv:2605.28532. <https://arxiv.org/abs/2605.28532>
10. Davidovich, Dotan, Yair Amar, Hai Rozenwajg, and Or Hiltch. 2026. *Permission Denied: Policy-Graded Evaluation of Coding Agents in Hardened Environments*. arXiv:2608.02670. <https://arxiv.org/abs/2608.02670>
11. Shayegani, Erfan, Keegan Hines, Yue Dong, Nael Abu-Ghazaleh, Roman Lutz, Spencer Whitehead, Vidhisha Balachandran, Besmira Nushi, and Vibhav Vineet. 2025. *Just Do It!? Computer-Use Agents Exhibit Blind Goal-Directedness*. arXiv:2510.01670. <https://arxiv.org/abs/2510.01670>
12. Yee, Ka-Ping. 2002. "User Interaction Design for Secure Systems." In *Information and Communications Security*, LNCS 2513, 278-290. Springer. <http://zesty.ca/pubs/icics-2002-uidss.pdf>
13. collusion.wiki. 2026. *Discovery of a New OpenAI Agent Message Board*. <https://collusion.wiki/>

Appendix A Limitations and Dual Use Considerations

Limitations.

1. *No base rate.* We cannot say how often evaluation runs contain a provisioning defect. The closest published statement is UK AISI's "in a number of runs", with no denominator. A search across AISI, METR, NIST CAISI, Anthropic's 141,006-run review and METR's task standard found no published denominator.
2. *Selection on the outcome.* The thirteen-case corpus was built by searching for escape events. It is consistent with the concentration claim and cannot test it.
3. *Single coder.* No second coder independently labelled the corpus, so we cannot report inter-rater reliability. Each judgement includes its source text, locator and coding basis.
4. *Mechanism of impossibility unknown.* OpenAI states that tasks were impossible but does not say why. If they were merely hard rather than under-provisioned, our framing applies to demonstrated solvability rather than resource availability.
5. *Toy experiments.* Both studies use one arithmetic task and five trials per model-condition cell. Hosted endpoints retained standard safety behaviour. The pilot had a cost-free abstention path and no proxy surface; no model used its shortcut. The follow-up used a non-networked proxy simulator and was designed after seeing the null result. Its GLM-only signal is post-hoc, imprecise and partly driven by a near-miss URL, so neither study estimates a transferable effect size.

Dual use. The incident record identifies blocked task completion as a context in which boundary-pushing concentrated. That observation could inform attempts to elicit similar behaviour from deployed agents. We judge disclosure worthwhile because the incidents are already public and the proposed defence is available to evaluators. This report therefore contains no exploit details or installation recipe. The harness uses a synthetic arithmetic task, never executes prohibited actions, has no public-network access and contains no real secrets.

LLM Usage Statement

We used GPT deep research to identify candidate sources, Claude to audit claims and develop the harness, and OpenAI Codex to re-audit the repository, test the code, correct source-path and report-language errors, and prepare the report in the official template. Team members checked the quotations against primary documents, corrected one dataset coding (C11), and reviewed the final claims and conclusions.